



Island Viewer

Présentation de notre projet de Prog&Algo (semestre 2).

Sommaire

- [Sommaire](#)
- [I - Plateforme et structures](#)
- [II - Liste des tâches](#)
- [III - Informations supplémentaires](#)
- [IV - Rapport](#)
 - 1. Choix algorithmiques
 - Color map
 - Masque
 - ImGui local
 - Bruit fractal
 - Poisson Disk Sampling
 - 2. Paramètres et impact visuel
 - Color map
 - Aléatoire
 - Intervalle de spawn
 - Bruit fractal
 - Poisson Disk Sampling
 - 3. Difficultés et solutions
 - a) Le Simplex qui marchait pas
 - Le problème
 - La solution

- b) Les objets qui font n'importe quoi
 - Le problème
 - La solution
- c) L'interface pas ergonomique
 - Le problème
 - Solution
- d) L'échec de l'ajout du Diamond Square (et des bruits matriciels)
 - Avant
 - Le problème
 - Ce qu'on a fait
 - Pourquoi c'est pas idéal
- e) Poisson Disk Sampling
 - Le problème : mauvaise implémentation de l'algorithme
 - La solution : demander des explications à Jules FOUCHY et réécrire le code étape par étape
 - Le problème : le temps d'exécution
 - La solution : implémenter la version de Bridson
 - Le problème : la confusion avec les x et y
 - La solution : besoin de clarté
- 4. Captures d'écrans comparatives
- 5. Post-mortem
 - Les problèmes et les résolutions
 - Avec plus de temps
 - Répartition du travail

I - Plateforme et structures

Le projet a été développé sur Windows et Linux (Ubuntu) à l'aide de Microsoft Visual Studio Code et JetBrains CLion

Configuration de Matthieu :

- Ubuntu 24.04.4 LTS 64bits
 - CLion
- Microsoft Windows 11 Home 25H2 (26200.8457) 64bits
 - CLion 2025.2.3 (Build #CL-252.26830.83, built on October 1, 2025)
- CPU : AMD Ryzen 5 7535HS, RAM : 16Go, GPU : NVIDIA GeForce RTX 4050 Laptop GPU (6 GB)

Configuration de Nils :

- Windows 11
 - Visual Studio Code

```

├── bin : exécutables
├── build : fichiers de compilation
├── img : images du rapport
├── imgui : librairie d'interface, en local
├── resources : ressources, comme les images de color map
├── src : code source
│   ├── utils : dossier avec les utilitaires
│   ├── app : fichiers avec les paramètres de l'application
│   ├── draw : fichiers de rendu d'interfaces et de la carte
│   ├── generation : fichiers de génération du terrain et des objets
│   ├── main.cpp : fichier d'exécution principal
│   └── noise : fichiers pour la génération des bruits

```

II - Liste des tâches

🔨 : En cours ✅ : Fini ❌ : À faire 🌊 : Amélioration supplémentaire ✅ : Amélioration de l'énoncé

Fait	Catégorie	En +	Tâche	Note	Qui
✅	-		Bruit fractal (FBM)		Matthieu
✅	-		Générer heightmap + couleur		Nils
✅	-		Poisson disk sampling (2D) + placement en 3D		Matthieu
✅	💻 IHM		Regénérer la heightmap		Nils
✅	💻 IHM		Regénérer le mesh 3D		Nils
✅	💻 IHM		Regénérer le poisson disk sampling		Matthieu
✅	💻 IHM		Paramètres (seed, size, height range)		Nils
❌	-	✅	Colormap builder		-
❌	-	✅	Coloration avancé (ex: en fonction de la pente, type de biome, etc)		-
✅	-	✅	Algorithmes différents bruit (Simplex, Worley, etc)		Nils

Fait	Catégorie	En +	Tâche	Note	Qui
✓	-	✓	Pile d'Algorithmes de bruit		Nils
✓	-	✓	Variation placement d'objet (taille, rotation)		Nils
✓	-	✓	Placement d'objet avec des conditions (pentes, hauteur)		Nils
✗	-	✓	Importer un modèle 3D (libre de droits) à la place du cube		-
✗	-	✓	Liste de modèles 3D (libre de droits) à placer avec conditions		-
✗	-	✓	Biomes (colormap, liste d'objets, influence sur la hauteur, bruit)		-
✗	-	✓	Connecter les îles avec des ponts, ou par la terre		-
✗	-	✓	Génération de différentes formes d'îles (au moins 3)		-

III - Informations supplémentaires

Extension pratique pour naviguer dans des endroits du code sur VS Code :

[Todo Tree](#)

Avec NOTE: et SOURCE: d'ajoutée dans [todo-tree.general.tags](#)

Il est aussi recommandé d'utiliser l' Outline dans le panneau de droite de VS Code

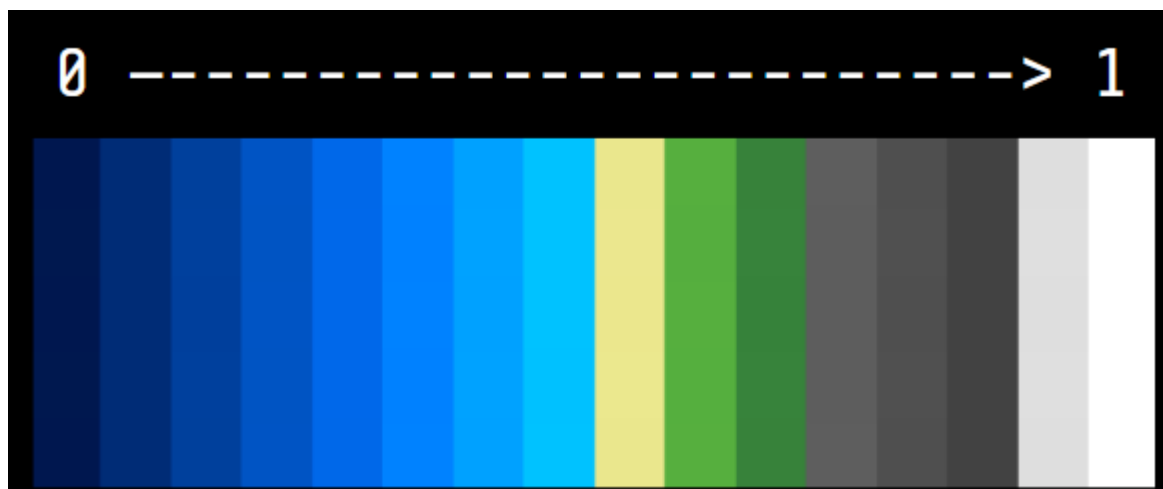
Il y a des fichiers qui ont des indentations bizarres à cause d'un formatage qui s'est mal passé (en faisant Ctrl + P et puis format document)

IV - Rapport

1. Choix algorithmiques

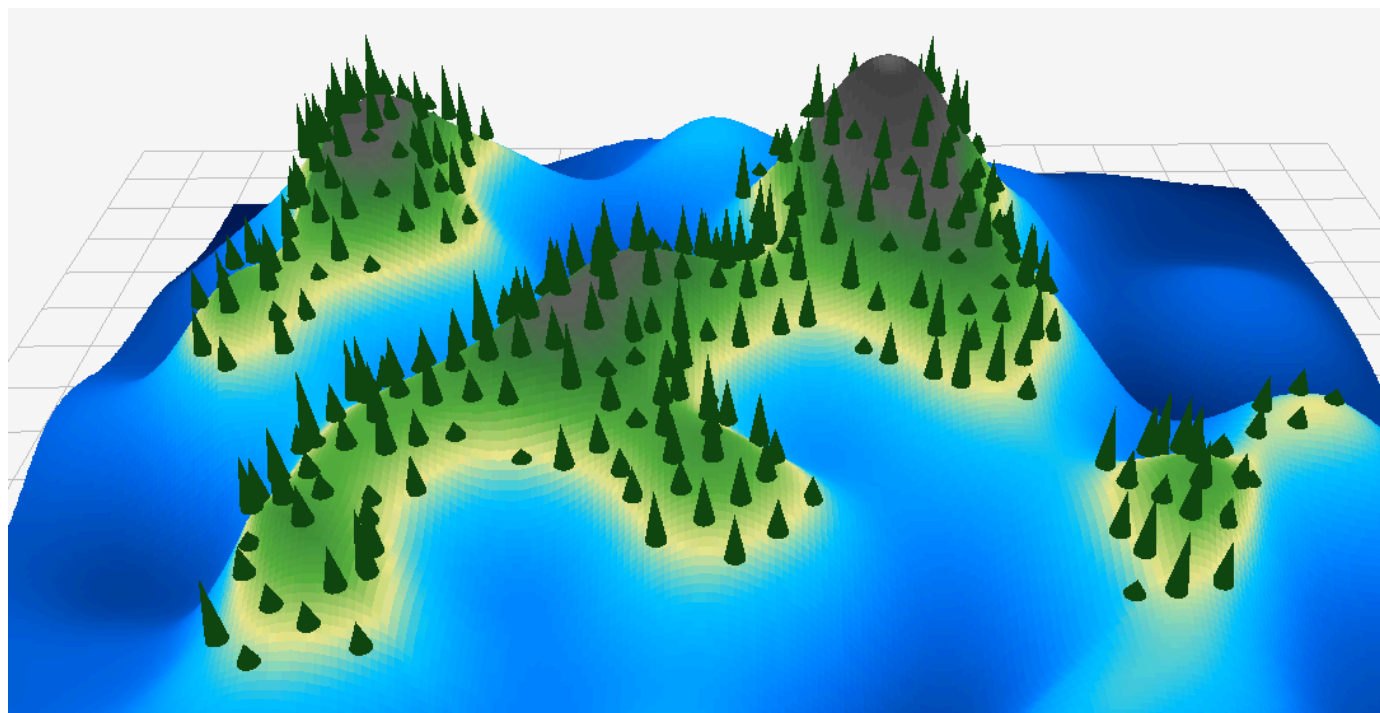
Color map

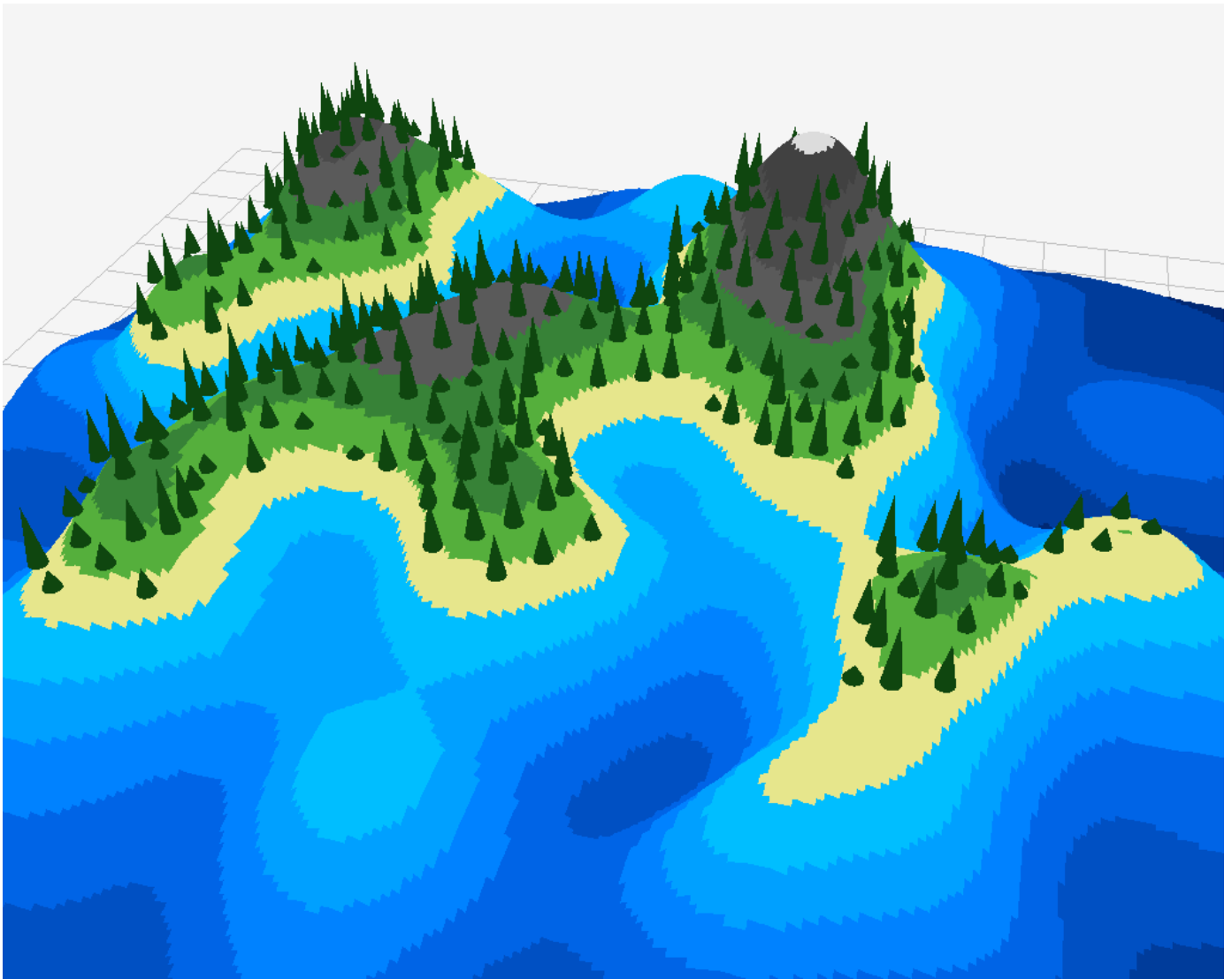
Pour la coloration de la carte, j'ai (Nils) décidé de faire une color map (palette de coloration) d'après une image comme j'avais fait pour le ★★★★★ Diamond Square pendant le Workshop de Prog Algo 1 qui mappe une valeur de la heightmap (0-1) à un pixel de la color map :



avec une interpolation linéaire (qui peut se désactiver) si la valeur ne correspond pas parfaitement à un pixel

Avec lerp





Masque

Un masque sélectionnable a été choisi pour générer une île de manière personnalisable, le meilleur masque restant quand même Gaussien (pour avoir une forme circulaire)

Cela a été décidé car nous avons plusieurs bruits à disposition alors pourquoi pas en faire une fonctionnalité pour à la fois la pile de bruit et le masque.

Bien que cela ne soit que très peu utile

ImGui local

La librairie ImGui est stocké en local car, il y avait besoin d'un objet d'interface `slider-range2` qui n'était pas disponible mais déjà développé par quelqu'un.

La conséquence c'est que le projet est un poil plus gros sur GitHub

(PS: le fetch-content dans le CMake ne marchait pas sur ce repo pour une raison obscure)

Bruit fractal

J'ai (Matthieu) fais le plus simple, j'ai effectué le bruit fractal en choisissant le bruit Perlin comme bruit de base.

Poisson disk sampling

Concernant le Poisson disk sampling, j'avais commencé à créer une version sans tableau (qui n'est donc pas la version de Bridson's).

Seulement, la version avec tableau (version de Bridson's) permet à l'algorithme de s'exécuter plus rapidement, c'est la version la plus populaire et la plus recommander, alors j'ai fini par l'implémenter (sachant qu'elle est demandé dans le sujet du projet).

2. Paramètres et impact visuel

Color map

Comme dit précédemment, il a été choisi de permettre une sélection de la palette de couleur de la carte (et il est facile d'en ajouter une) pour avoir plusieurs types de "biomes"

Aléatoire

Il a été décidé d'ajouter des paramètres sur le placement aléatoire des objets pour faire varier la taille et l'orientation pour avoir de la variation sur les objets, et avoir un rendu plus proche de la vie réelle (c'est-à-dire les arbres ne sont pas tous de la même taille)

Intervalle de spawn

Un intervalle de hauteur de spawn autorisé a été codé pour permettre une condition de placement pour, par exemple, éviter d'avoir des objets trop haut dans les montagnes, ou dans la mer.

Bruit fractal

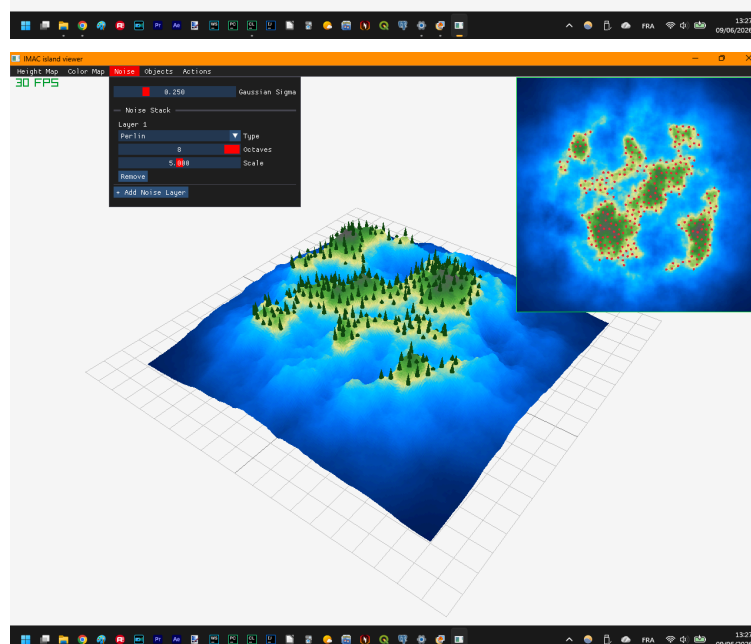
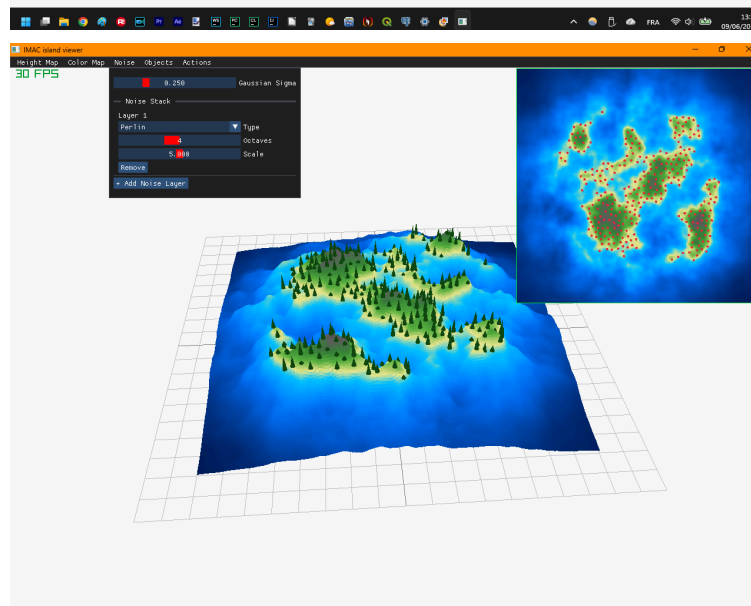
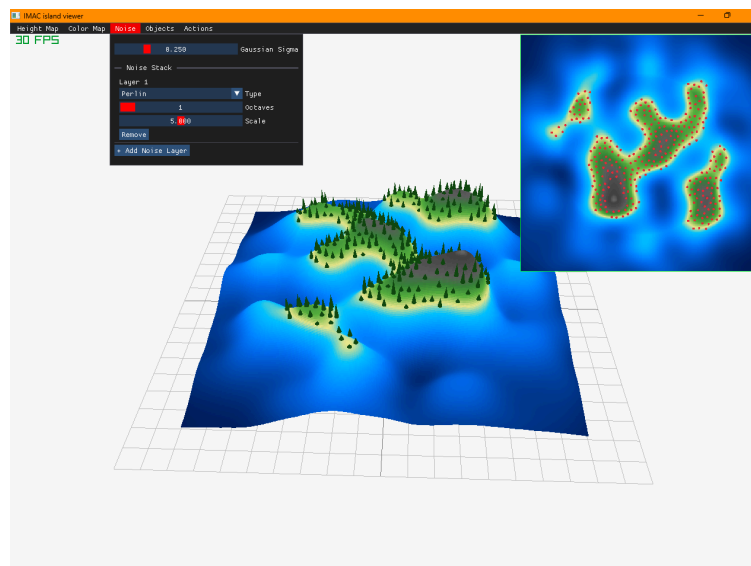
Concernant le bruit fractal, ses paramètres sont complexes.

Voici une description des paramètres venant de l'article <https://thebookofshaders.com/13/?lan=fr> : *En ajoutant différentes itérations du bruit (différentes octaves), dont on augmente la fréquence (la lacunarité) et dont on réduit l'amplitude (le gain), on obtient une granularité qui nous permet de préserver les détails fins d'un bruit.*

J'ai (Matthieu) estimé qu'un nombre correct d'octaves est 6, c'est un bon nombre, cela permet d'avoir une carte suffisamment détaillée sans que cela prenne trop de temps pour la génération.

Nous pouvons diminuer ce nombre pour constater des différences, cependant, si on l'augmente, les modifications seront quasiment invisible.

Par défaut, nous avons laissé cette valeur à 1, mais grâce à l'interface, nous pouvons la changer. Voici un comparatif :



De gauche à droite les valeurs d'octaves sont : 1, 4, 8

Pour tester cette différence il suffit sur l'interface :

1. Dans le menu Height Map -> désactiver Random Seed
2. Dans le menu Noise -> faire varier le nombre d'octaves
3. Dans Actions -> faire "Regenerate All"

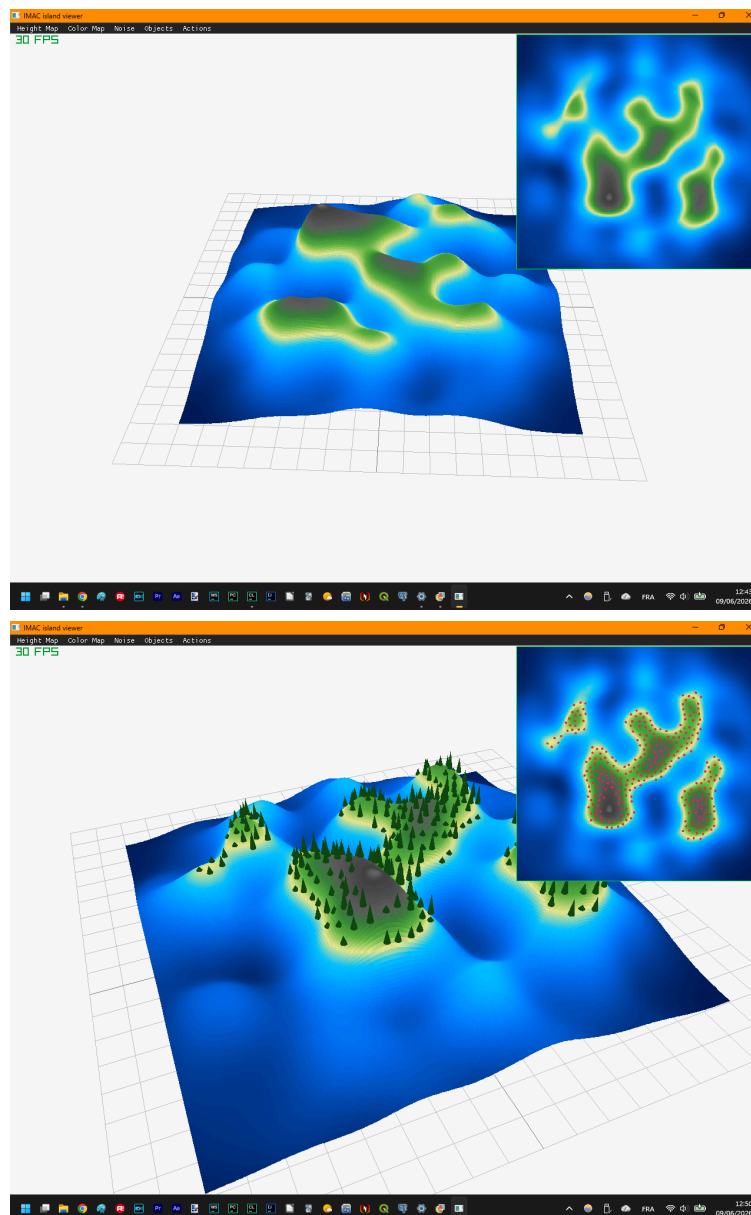
À noter que même si j'ai (Matthieu) travaillé à partir du bruit de Perlin, l'interface propose d'autres types de bruit.

Poisson disk sampling

La taille de notre carte varie de 0 à 1 selon notre code, contrairement au code des vidéos partagé dans le sujet du rapport ou des articles trouvables sur internet.

De ce fait, le paramètre radio "params.poissonRadius" par exemple ne peut pas être de 1 puisqu'il indique le rayon entre les points (pour résumer, dans les faits, ce n'est pas exactement ça).

J'ai (Matthieu) choisis 0,2 comme valeur, car cela me paraissait à la fois suffisamment grand pour que les points ne soient pas collés, et suffisamment petit pour qu'il y ait assez de points.



Radius de 1 à gauche, radius de 0.2 à droite

De même, dans la vidéo de Sebastian Lague, il utilise une variable appelée `sampleRegionSize` (ce qui correspond à la taille de carte), sauf que dans notre cas, il nous suffit d'indiquer juste 1.

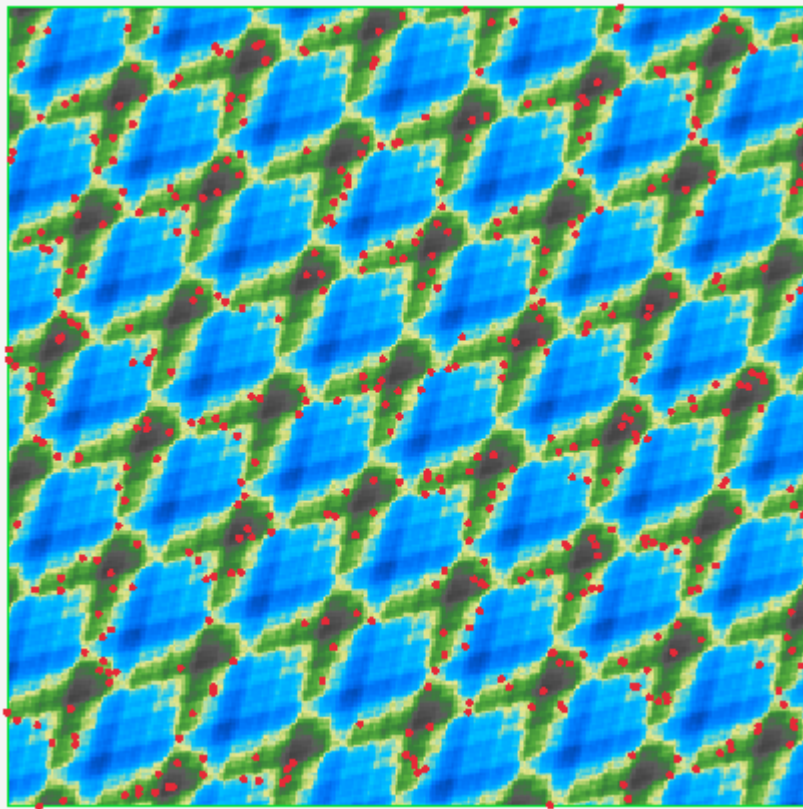
3. Difficultés et solutions

a) Le Simplex qui marchait pas

Le problème

Pour le Simplex Noise, nous avons repris le code de l'article [Simplex noise demystified](#)

Cependant après avoir adapté le code, il y avait un problème : le bruit se répétait en boucle



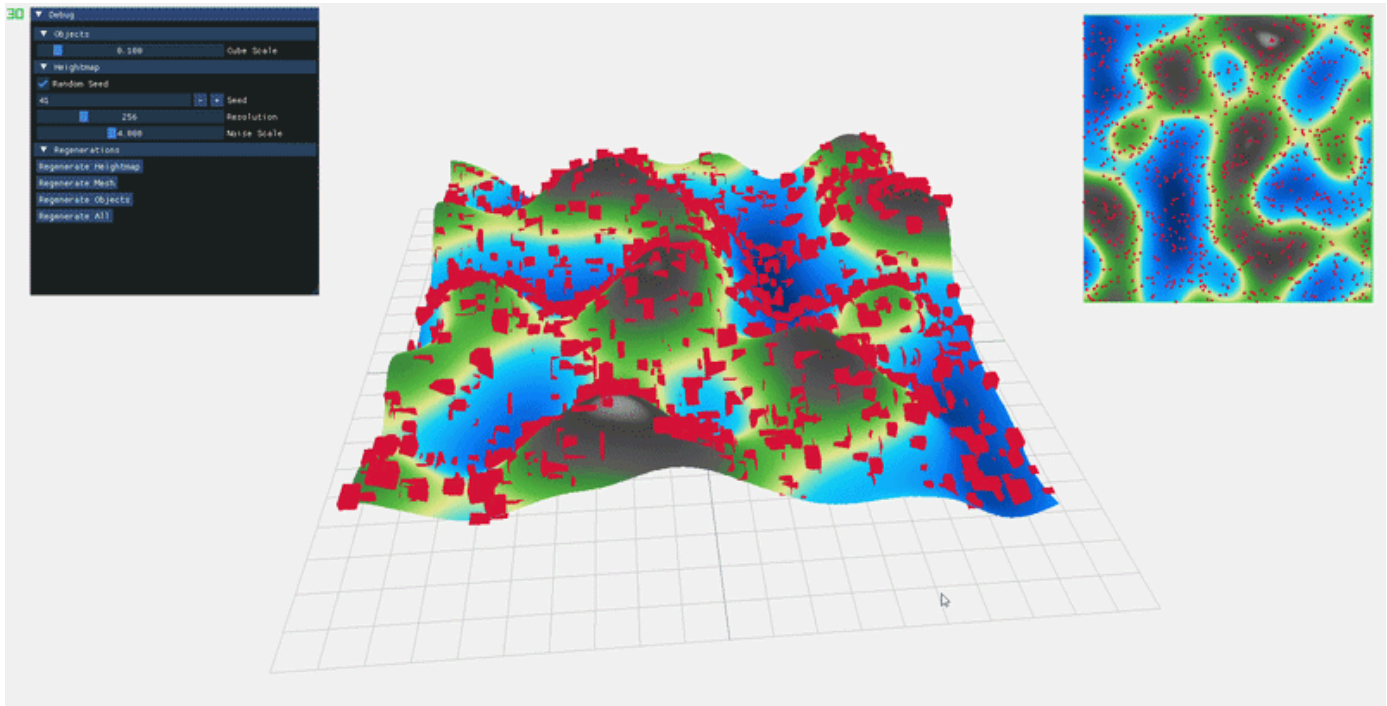
La solution

Après avoir crié à l'aide au chargé de TD, qui n'a pas trouvé la solution car le code était une exacte copie de ce que l'article avait, la solution était toute simple, malgré des heures de tirage de cheveux : le code n'initialisait pas la liste de permutations (ce qui fait en sorte que le bruit est un bruit)

b) Les objets qui font n'importe quoi

Le problème

à un moment du projet, le placement des objets pouvait être aléatoire. Cependant ces valeurs aléatoires changeaient à chaque frame car la fonction d'aléatoire `randF` était appelée pendant le rendu.



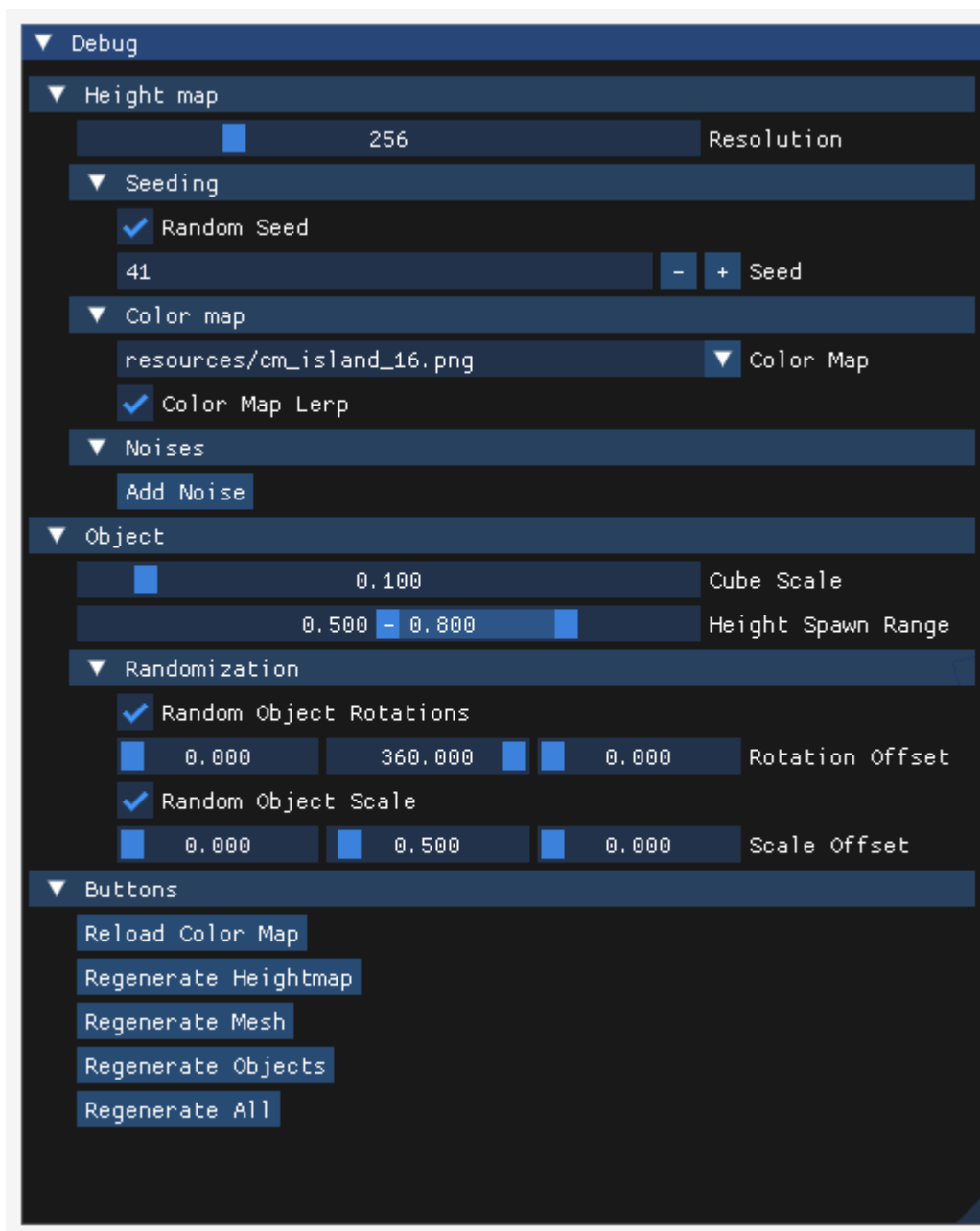
La solution

À la construction de la carte et du placement des points, les données aléatoires de placement ont été stockées dans une liste de `ObjectRandomizationData` disponible dans le `context` de l'application. Cette structure contient la modification de la position et la rotation de base de l'objet, pour chacun. Résultat : on récupère la donnée générée et stockée bien avant, ce qui ne la change pas à chaque frame.

c) L'interface pas ergonomique

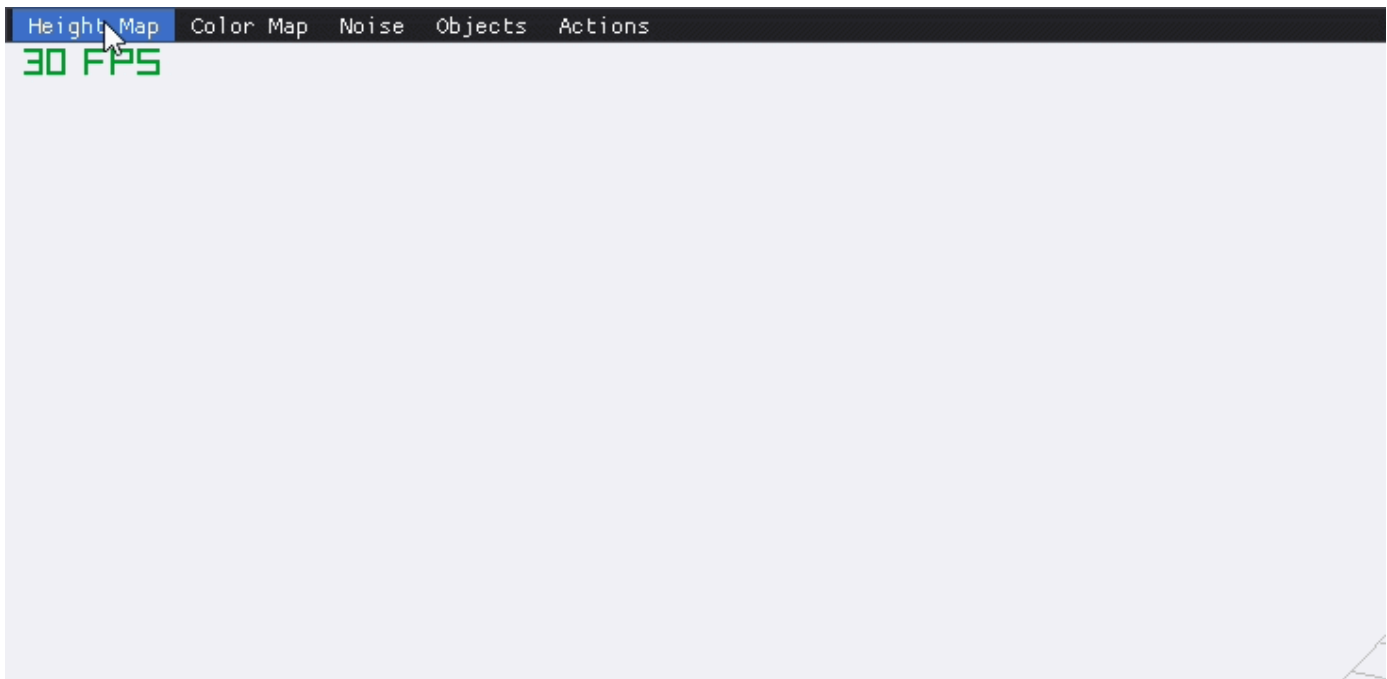
Le problème

Avant, il y avait uniquement une fenêtre avec tous les morceaux d'interface listés dedans, un peu séparés les uns des autres mais rien de vraiment visible. Et à mesure que le projet progressait il y avait beaucoup de choses. D'autant plus qu'il y avait la pile de bruit à gérer qui ajoutait beaucoup de lignes



Solution

Il a donc été décidé de refaire l'interface sous forme de sous-menus avec des couleurs différentes et des sections propres à chaque catégorie de paramètres



d) L'échec de l'ajout du Diamond Square (et des bruits matriciels)

Le code de l'échec est dans la branche [diamond-square-attempt](#)

Avant

La noise stack était simple : chaque bruit est une fonction qui prend une position et une seed, et renvoie un float.

```
struct Noise {
    std::function<float(glm::vec2 const&, int)> func;
    int nbOctave {6};
    float scale {5.f};
};

struct ImageGenerationData {
    ...
    static std::function<float(glm::vec2 const&, int)> noiseFunctions[2];
};

struct ImageGenerationParameters {
    ...
    std::vector<Noise> noiseStack {};
};
```

Le problème

Le Diamond Square est un algorithme **matriciel** : il génère une heightmap entière (`Image`) d'un coup, plutôt que de renvoyer une valeur par position comme `Perlin` et `Simplex` . Il fallait donc supporter deux types de bruits dans la même stack :

- **Fonction** : `float(glm::vec2, int)` , point par point (Simplex, Perlin...)
- **Matrice** : `Image( params de l'algo )` , qu'on sample ensuite (mais chaque algo matriciel peut avoir une signature entrante différente)

Ce qu'on a fait

Pour garder une struct `Noise` unique, on a utilisé un `std::variant` pour le champ `func`, avec une enum `NoiseType` pour savoir lequel des deux types est actif.

```
enum NoiseType {
    FUNCTION,
    MATRIX,
};

using NoiseFunction = std::variant
    std::function<float(glm::vec2 const&, int)>, // PERLIN + SIMPLEX
    std::function<Image(float)>                // DS
    //les autres algo matrice de type Image( leurs paramètres )
>;

struct Noise {
    NoiseType type;
    NoiseFunction func;
    int nbOctave {2};
    float scale {5.f};
};

struct ImageGenerationData {
    ...
    static std::variant
        std::function<float(glm::vec2 const&, int)>,
        std::function<Image(float)>
    > noiseFunctions[3];
    const NoiseType noiseFunctionsTypes[2] = {
        NoiseType::FUNCTION,
        NoiseType::MATRIX
    };
};

struct ImageGenerationParameters {
    ...
    std::vector<Noise> noiseStack {};
};

struct App {
    ...
    std::vector<Image> noiseMatrixStack {}; // résultats des algos matriciels
    ...
};
```

Pourquoi c'est pas idéal

- Le variant ne peut pas couvrir toutes les signatures possibles des algos matriciels : il faudrait ajouter à la main chaque nouvelle signature dans `NoiseFunction`
- Il faut jongler avec `std::get` / `std::get_if` partout où on utilise un bruit
- L'interface `ImGui` doit s'adapter manuellement à chaque type de bruit (ou alors on s'adapte à la signature de la fonction de bruit matriciel)
- Une `noiseMatrixStack` séparée dans `AppContext` stocke les résultats des algos matriciels, ce qui crée une désynchronisation potentielle avec la `noiseStack`

une option plus simple aurait été de générer la matrice pour chaque position au lieu de tout stocker une fois, mais niveau performance c'est catastrophique

e) Poisson Disk Sampling

Le problème : mauvaise implémentation de l'algorithme

Mon principal problème était un temps d'exécution très long et le fait de ne pas savoir si l'algorithme fonctionnait bien. Mais ce problème était lié au fait que je n'implémentais pas la version de Bridson's, je voulais aller vite et que j'avais mal appréhendé le problème.

La solution : demander des explications à Jules FOUCHY et réécrire le code étape par étape

L'une des premières pistes était de demander de l'aide à Jules FOUCHY, notamment pour me réexpliquer certains points de l'algorithme (tel qu'afficher les points candidats de manière cyclique autour du point actif et non dans un carré), de regarder des versions d'algorithme dans d'autres langages, mais surtout de reconstruire l'algorithme étape par étape.

Reconstruire étape par étape c'était ce que faisait The Coding Train dans sa vidéo :

- J'ai commencé à afficher un point aléatoire sur la carte
- Affiché les points candidats autour de ce point
 - Ici, je me suis aperçus que l'un des pourquoi cela ne marchait pas, c'était le radius trop grand. Cela m'a permis de réajuster la valeur de la variable proprement.

J'ai ainsi pu constater le bon fonctionnement de l'algorithme à ce stade, et continuer progressivement.

Le problème : le temps d'exécution

Parce que je n'utilisais pas la version de Bridson, plutôt que de m'aider d'un tableau pour comparer un point candidat avec les points périphérie, je comparais le point candidat avec TOUS les points de la carte en parcourant une liste. A petite échelle, ça allait, à grande échelle, cela prenait tellement de temps, que je pouvais difficilement constater le résultat.

La solution : implémenter la version de Bridson

Et donc, remplacer ma liste par un tableau où les points d'une certaine zone se retrouve dans une même case.

Si on constate que la case du tableau associé à notre zone comporte déjà un point (car marqué 0 et non plus -1 (-1 signifiant que c'est vide)), nous ne pouvons pas placer notre point candidat (la zone est déjà occupée).

C'est plus complexe à mettre en place, mais l'autre méthode n'était viable.

Le problème : la confusion avec les x et y

Avec la version de Bridson, je manipule un tableau (donc on utilise x et y), mais également des points sur la carte (on utilise aussi des x et y), et on compare des points sur la carte, dans les listes, dans le tableau, en d'autre terme, il y a plusieurs x et y dans l'algorithme, mais chacune ont un rôle différent.

De ce fait, il est très simple de confondre des variables notamment quand on regarde le code de quelqu'un d'autres (articles sur internet ou vidéos sur internet).

La solution : besoin de clarté

La solution a été de donner des noms de variable explicite comme `candidatGrilleX`, d'utiliser directement `pointCandidat.x` sans les stockés dans des variables, mais aussi et surtout, de commenter un maximum. C'est-à-dire, ce que ça représente, à quoi cela correspond, même si cela peut paraître évident. Parce qu'il est beaucoup trop facile de comprendre une partie évidente de l'algorithme, et d'avoir un doute. Alors le fait de commenter même les évidences, c'est un moyen d'être sûr d'avoir tout en tête pour bien comprendre ce que l'on fait.

4. Captures d'écrans comparatives

Nous n'avons pas de capture intéressante à montrer hormis celles déjà présente dans ce rapport.

5. Post-mortem

- qu'est-ce qui a bien fonctionné, quels ont été les problèmes rencontrés 
- comment vous les avez surmontés, et ce que vous auriez fait différemment. 
- Avec plus de temps, qu'est-ce que vous pourriez ajouter ? 
- Comment s'est passée la répartition du travail dans le groupe ? 

Les problèmes et les résolutions

Les problèmes ont été rencontrés sur plusieurs parties du projet, et avec une complexité différente. La plupart du temps, la solution était trouvée grâce à internet, notre cerveau, grâce au chargé de TD ou en message privé avec le chargé de projet (□ toi).

Avec plus de temps

Avec plus de temps, en témoigne la liste de tâches, nous aurions pu implémenter un outil de création de color map plutôt que de charger une image, et ce avec [imgui_gradient](#) de Coollab

Nous aurions pu faire comme dans Minecraft et mettre des biomes selon un set d'objets 3D, une color map, et placés selon une carte de températures

Nous aurions pu ajouter plus de bruits, pour avoir des formes d'îles comme un croissant, un donut, des carrés, etc...

Et avec **ÉNORMÉMENT** de temps et de détermination, reproduire [WorldBox](#) en 3D (il faut savoir être ambitieux dans la vie)

Répartition du travail

La répartition a été faite de sorte à ce que les choses déjà manipulées par Nils pendant le workshop de prog algo (avec le Diamond Square) et le [Workshop d'esthétique et algorithmique](#) soient faites par lui pour garantir une efficacité sur le reste du projet. Matthieu a été le plus en charge des fonctionnalités nécessaires du projet (çàd la base) pour garantir un code compréhensible.

Pendant ce temps Nils faisait aussi des bonus sur des choses plus complexes et expérimentales (comme l'échec cuisant du Diamond Square).

Cela correspondait bien aux deux membres car les charges de travail sur les autres projets n'étaient pas les mêmes pour l'un et l'autre